# CS 39006: Computer Networks Laboratory

# POSIX Network Programming: UDP Sockets


Department of Computer Science and Engineering

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

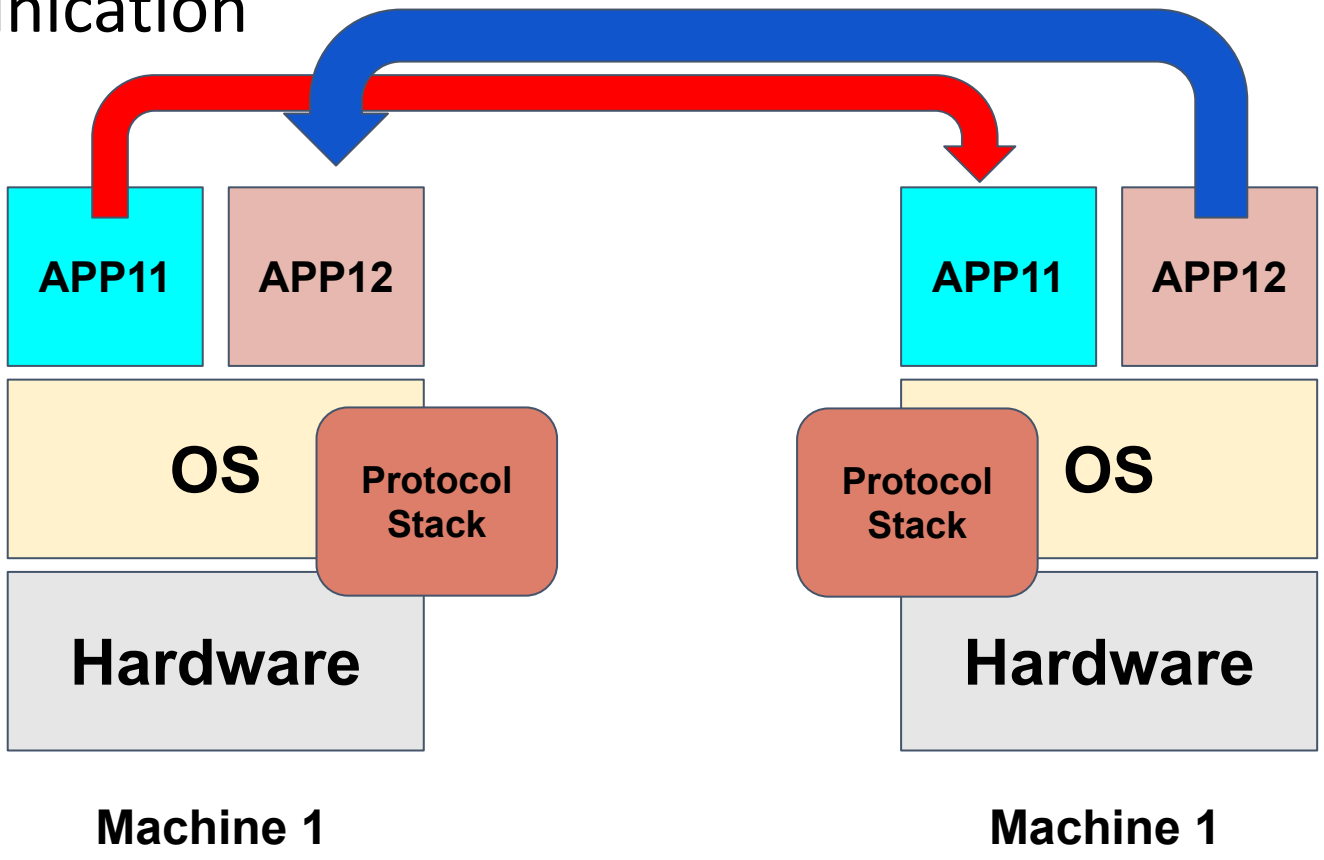
Abhijnan Chakraborty
abhijnan@cse.iitkgp.ac.in

Sandip Chakraborty
sandipc@cse.iitkgp.ac.in

# Overview

## Why Sockets?

- Enable process-to-process communication
- Basis of:
  - Web (HTTP)
  - DNS
  - Streaming
  - Multiplayer games
  - IoT & sensor systems




## Today's Focus

- POSIX sockets fundamentals
- UDP (datagram) sockets
- Writing a UDP client and server in ANSI C

# What Are POSIX Sockets?

## Definition

- **POSIX sockets** = standardized API for network communication
- Same interface across:
  - Linux
  - macOS
  - UNIX-like systems

# What Are POSIX Sockets?

## Definition

- **POSIX sockets** = standardized API for network communication
- Same interface across:
  - Linux
  - macOS
  - UNIX-like systems

**Key Idea:**

A **socket** is an endpoint for communication

# What Are POSIX Sockets?

## Definition

- **POSIX sockets** = standardized API for network communication
- Same interface across:
  - Linux
  - macOS
  - UNIX-like systems

**Key Idea:**

A **socket** is an endpoint for communication

## Abstraction

- File-descriptor–like interface
- Uses standard system calls

# Socket Communication Model

## Components

- **IP Address** → identifies host
- **Port Number** → identifies application
- **Protocol** → TCP or UDP

# Socket Communication Model

## Components

- **IP Address** → identifies host
- **Port Number** → identifies application
- **Protocol** → TCP or UDP

## Socket Address

```
(IP address, Port, Protocol)
```

# Socket Communication Model

## Components

- **IP Address** → identifies host
- **Port Number** → identifies application
- **Protocol** → TCP or UDP

## Socket Address

(IP address, Port, Protocol)

## Typical Flow

**Process → Socket → Network → Socket → Process**

# TCP vs UDP

| Feature | TCP | UDP |
|---|---|---|
| Connection | Connection-oriented | Connectionless |
| Reliability | Guaranteed | Best-effort |
| Ordering | Yes | No |
| Overhead | High | Low |
| Speed | Slower | Faster |

# TCP vs UDP

| Feature | TCP | UDP |
|---|---|---|
| Connection | Connection-oriented | Connectionless |
| Reliability | Guaranteed | Best-effort |
| Ordering | Yes | No |
| Overhead | High | Low |
| Speed | Slower | Faster |

**When to Use UDP?**

- Real-time systems
- DNS
- Video/Audio streaming
- Sensor data, telemetry

# UDP (Datagram) Sockets

**Key Characteristics**

- No connection setup
- Each message is an independent datagram
- Application handles:
  - Loss
  - Duplication
  - Ordering

# UDP (Datagram) Sockets

## Key Characteristics

- No connection setup
- Each message is an independent datagram
- Application handles:
  - Loss
  - Duplication
  - Ordering

## POSIX Socket Type

```
SOCK_DGRAM
```

# UDP Socket API (High-Level)

## Core System Calls

- `socket()`
- `bind()`
- `sendto()`
- `recvfrom()`
- `close()`

# UDP Socket API (High-Level)

## Core System Calls

- `socket()`
- `bind()`
- `sendto()`
- `recvfrom()`
- `close()`

## Header Files

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
```

# Creating a UDP Socket

```
int sockfd;
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

**Parameters**

- AF_INET → IPv4
- SOCK_DGRAM → UDP
- 0 → default protocol (UDP)

# Creating a UDP Socket

```
int sockfd;
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

**Parameters**

- AF_INET → IPv4
- SOCK_DGRAM → UDP
- 0 → default protocol (UDP)

**Error Handling**

```
if (sockfd < 0) {
    perror("socket failed"); }
```

# Socket Address Structure

```
struct sockaddr_in server_addr;
```

**Fields:**

```
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY;
```

# Socket Address Structure

```
struct sockaddr_in server_addr;
```

**Fields:**

```
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY;
```

**Why `htons()`?**

- **Converts host byte order → network byte order**

# UDP Server: Binding the Socket

**Purpose**

- Associate socket with a port

```
bind(sockfd,
     (struct sockaddr *)&server_addr,
     sizeof(server_addr));
```

- **Error Check**

```
if (bind(...) < 0) {
    perror("bind failed");
}
```

# Receiving Data (UDP Server)

```
char buffer[1024];
struct sockaddr_in client_addr;
socklen_t len = sizeof(client_addr);

int n = recvfrom(sockfd,
                 buffer,
                 sizeof(buffer),
                 0,
                 (struct sockaddr *)&client_addr,
                 &len);
```

# Sending Data (UDP Server)

```
sendto(sockfd,
       "Hello Client",
       12,
       0,
       (struct sockaddr *)&client_addr,
       len);
```

**Note**

- Server replies using client's address from `recvfrom()`

# Minimal UDP Server (ANSI C)

```
int sockfd;
char buffer[1024];
struct sockaddr_in server_addr, client_addr;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY;
```

# Minimal UDP Server (ANSI C)

```
bind(sockfd,
     (struct sockaddr *)&server_addr,
     sizeof(server_addr));

socklen_t len = sizeof(client_addr);
recvfrom(sockfd, buffer, sizeof(buffer), 0,
         (struct sockaddr *)&client_addr, &len);

sendto(sockfd, "ACK", 3, 0,
       (struct sockaddr *)&client_addr, len);

close(sockfd);
```

# UDP Client Basics

**Differences from Server**

- No `bind()` required (OS assigns port)
- Directly uses `sendto()`

# UDP Client Code

```
int sockfd;
struct sockaddr_in server_addr;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

sendto(sockfd, "Hello Server", 12, 0, (struct sockaddr *)&server_addr,
       sizeof(server_addr));

close(sockfd);
```

# Common Pitfalls

- Forgetting `htons()` / `inet_pton()`
- Assuming delivery is guaranteed
- Buffer overflows
- Not checking return values
- Firewall blocking UDP ports